Python Engineering - EX24

WEEK 04 | CLASS 16 - Testing & Project Structure

Goal: close the bootcamp by building a small verifiable project structure. We will separate application code from tests, automatically check known results and distinguish a failed assertion from an unexpected runtime error.

> We will not install a testing framework yet. First we build the mechanism ourselves to understand **Arrange -> Act -> Assert**.

Exact path

```
01_python_bootcamp/
`-- week_04_oop_files_testing/
    `-- class_16_testing_project_structure/
        |-- notes/
        `-- exercises/
            `-- ex24_testing_project_structure/
                |-- vertical_state.py
                |-- file_io.py
                |-- main.py
                |-- run_tests.py
                |-- tests/
                |   |-- __init__.py
                |   |-- test_vertical_state.py
                |   `-- test_file_io.py
                `-- data/
```

What we reuse

Reuse clean versions of `VerticalState` and the File I/O functions from EX22/EX23. Remove exercise-only demonstration prints; here we want reusable application code.

File	Responsibility
vertical_state.py	Physical model and vertical state
file_io.py	Save/load trajectory CSV files
main.py	Application usage example
tests/*.py	Automated checks
run_tests.py	Execute and summarize all tests

Reference values

```
initial state:
h = 100.0 m
v = 20.0 m/s
t = 0.0 s
g = 9.81 m/s^2
dt = 0.1 s

after one Euler step:
h = 102.0
v = 19.019
t = 0.1
```

BLOCK A - Tests are normal functions

```
import numpy as np
from vertical_state import VerticalState


def test_initial_state():
    # ARRANGE
    state = VerticalState(h=100.0, v=20.0)

    # ACT
    result = state.state_vector()

    # ASSERT
    expected = np.array([100.0, 20.0, 0.0])
    assert np.allclose(result, expected), (
        f'Expected {expected}, got {result}'
    )
```

A test executes part of the program and compares actual behavior with expected behavior.

BLOCK B - Euler method test

```
def test_euler_step():
    state = VerticalState(h=100.0, v=20.0)
    dt = 0.1

    state.euler_step(dt)

    result = state.state_vector()
    expected = np.array([102.0, 19.019, 0.1])

    assert np.allclose(result, expected), (
        f'Expected {expected}, got {result}'
    )
```

BLOCK C - Why np.allclose() rather than ==

```
assert np.allclose(result, expected)
```

Floating-point calculations should usually be compared with small numerical tolerances rather than exact equality.

BLOCK D - File I/O round-trip test

```
from pathlib import Path
import numpy as np
from file_io import save_trajectory_csv, load_trajectory_csv


def test_csv_round_trip():
    base_dir = Path(__file__).resolve().parent.parent
    data_dir = base_dir / 'data'
    data_dir.mkdir(exist_ok=True)
    test_file = data_dir / 'test_round_trip.csv'

    original = np.array([
        [100.0, 20.0, 0.0],
        [102.0, 19.019, 0.1]
    ])

    save_trajectory_csv(test_file, original)
    loaded = load_trajectory_csv(test_file)

    assert original.shape == loaded.shape
    assert np.allclose(original, loaded)
    test_file.unlink(missing_ok=True)
```

BLOCK E - Test an expected exception

```
def test_missing_file_raises():
    base_dir = Path(__file__).resolve().parent.parent
    missing_file = base_dir / 'data' / 'does_not_exist.csv'

    try:
        load_trajectory_csv(missing_file)
    except FileNotFoundError:
        pass
    else:
        assert False, (
            'Expected FileNotFoundError, but no exception was raised'
        )
```

Correct behavior may be that invalid input raises the correct exception.

BLOCK F - __init__.py

Create an empty `tests/__init__.py`. For this exercise, simply understand that it lets `tests` be imported as a package by `run_tests.py`.

```
# tests/__init__.py
# empty file
```

BLOCK G - Build a tiny test runner

```
from tests.test_vertical_state import (
    test_initial_state,
    test_euler_step,
)
from tests.test_file_io import (
    test_csv_round_trip,
    test_missing_file_raises,
)

TESTS = [
    test_initial_state,
    test_euler_step,
    test_csv_round_trip,
    test_missing_file_raises,
]

def main():
    passed = failed = errors = 0

    for test in TESTS:
        try:
            test()
        except AssertionError as error:
            failed += 1
            print(f'FAIL  {test.__name__}')
            print(f'      {error}')
        except Exception as error:
            errors += 1
            print(f'ERROR {test.__name__}')
            print(f'      {type(error).__name__}: {error}')
        else:
            passed += 1
            print(f'PASS  {test.__name__}')

    print('\nSUMMARY')
    print(f'Passed: {passed}')
    print(f'Failed: {failed}')
    print(f'Errors: {errors}')

if __name__ == '__main__':
    main()
```

Here a broad `except Exception` is deliberate at the test-runner boundary: unexpected errors are reported as ERROR without stopping the remaining tests.

BLOCK H - Run from the exercise root

```
python run_tests.py

PASS  test_initial_state
PASS  test_euler_step
PASS  test_csv_round_trip
PASS  test_missing_file_raises

SUMMARY
Passed: 4
Failed: 0
Errors: 0
```

BLOCK I - Break something deliberately

```
expected = np.array([999.0, 19.019, 0.1])
```

Run the tests, observe FAIL, then restore the correct expected value. This proves that the test actually detects a regression.

BLOCK J - FAIL is not ERROR

Result	Meaning
PASS	Observed behavior matches the expectation
FAIL	Test executed, but an assertion was false
ERROR	Test could not complete because of an unexpected exception

FINAL VALIDATION - answer in comments

```
- What is a test?
- Application code vs test code?
- Explain Arrange, Act and Assert.
- What does assert do?
- What happens when an assertion is False?
- Why np.allclose for floating-point arrays?
- Why should each test check one clear behavior?
- What is a regression?
- Why should tests be repeatable?
- PASS vs FAIL vs ERROR?
- Why intentionally test FileNotFoundError?
- Why is except Exception acceptable in our test runner but not a good default everywhere?
- What is tests/__init__.py doing here?
- Why separate the tests folder?
- Why should main.py not contain test logic?
- What advantage does this structure provide before GitHub?
```

Completion criterion

EX24 is complete when all four tests pass, you can deliberately produce at least one FAIL, you understand FAIL vs ERROR, and you can explain why application code, data and tests are separated.